

Lab Assignment 7

Object-Oriented Programming

Branch

B.E., 2nd Year – ENC



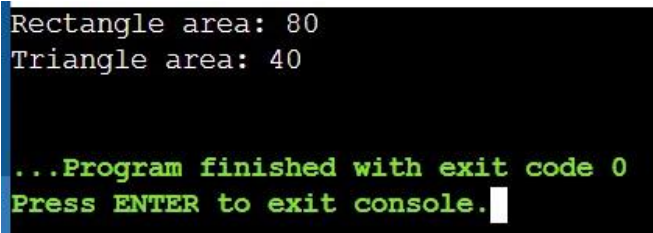
Submitted By:

Mridul Sharma

Roll No. 1024150204

Batch: 2023

Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated

CODE	TESTCASES
<pre> #include<iostream> using namespace std; class polygon { protected: int width, height; public: void set_value(int w, int h) { width = w; height = h; } }; class rectangle : public polygon { public: int calculate_area() { return width * height; } }; class triangle : public polygon { public: int calculate_area() { return width * height / 2; } }; int main() { polygon *p; rectangle r; triangle t; p = &r; p- >set_value(10, 5); </pre>	 <pre> Rectangle area: 80 Triangle area: 40 ...Program finished with exit code 0 Press ENTER to exit console. </pre>

```

        cout << "Rectangle area: " <<
r.calculate_area() << endl;
p = &t;      p->set_value(10,
5);      cout << "Triangle area:
" <<
t.calculate_area() <<
endl;      return 0;
}
}

```

Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

CODE

```

#include<iostream> using
namespace std;
class shape
{ public:
    virtual void area() {}
    virtual void display() {}
};
class circle : public shape {
float r; public:
    circle(float x) { r = x; }
void area() {
    cout << "Area of circle: " << 3.14 * r * r << endl;
}
    void display() {
        cout << "Shape: Circle" << endl;
    }
}

```

```

    }
};
class rectangle : public shape {
float l, b; public:
    rectangle(float x, float y) { l = x; b = y; }
void area() {
    cout << "Area of rectangle: " << l * b << endl;
}
void display() {
    cout << "Shape: Rectangle" << endl;
}
};
class triangle : public shape {
float b, h; public:
    triangle(float x, float y) { b = x; h = y; }
void area() {
    cout << "Area of triangle: " << 0.5 * b * h <<
endl;    }
void display() {
    cout << "Shape: Triangle" << endl;
}
}; int main() {
shape *s;    circle
c(5);    rectangle
r(4, 6);    triangle
t(3, 7);
    s = &c; s->display(); s-
>area();    s = &r; s->display();
s->area();    s = &t; s->display();
s->area();

    return 0;
}

```

TESTCASE

```

Shape: Rectangle
Area of rectangle: 10
Shape: Triangle
Area of triangle: 17.5

...Program finished with exit code 0
Press ENTER to exit console.

```

Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

CODE	TESTCASES
<pre> #include<iostream> #include<math.h> using namespace std; float area(float b, float h) { return 0.5 * b * h; } float area(float a) { return (1.732 / 4) * a * a; } float area(float a, float b, float c) { float s = (a + b + c) / 2; return sqrt(s * (s - a) * (s - b) * (s - c)); } int main() { cout << "Right triangle: " << area(5, 6) << endl; cout << "Equilateral: " << area(9) << endl; </pre>	<pre> Right triangle: 15 Equilateral: 35.073 Isosceles: 20.3332 ...Program finished with exit code 0 Press ENTER to exit console. </pre>

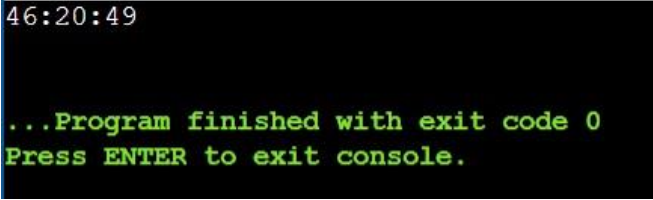
<pre> cout << "Isosceles: " << area(6, 7, 8) << endl; return 0; } </pre>	
---	--

Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

CODE	TESTCASES
<pre> #include<iostream> using namespace std; class Student { public: virtual void show() = 0; }; class Engineering : public Student { public: void show() { cout << "Engineering student" << endl; } }; class Medicine : public Student { public: void show() { cout << "Medicine student" << endl; } }; class Science : public Student { public: </pre>	<pre> Engineering student Medicine student Science student ...Program finished with exit code 0 Press ENTER to exit console. </pre>

<pre> void show() { cout << "Science student" << endl; } }; int main() { Student *s[3]; Engineering e; Medicine m; Science sc; s[0] = &e; s[1] = &m; s[2] = &sc; for(int i = 0; i < 3; i++) s[i]->show(); return 0; } </pre>	
---	--

Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

CODE	TESTCASES
<pre> #include<iostream> using namespace std; class Time { int h, m, s; public: Time() {} Time(int a, int b, int c) { h = a; m = b; s = c; } Time operator+(Time t) { </pre>	 <pre> 46:20:49 ...Program finished with exit code 0 Press ENTER to exit console. </pre>

<pre> Time temp; temp.s = s + t.s; temp.m = m + t.m + temp.s / 60; temp.s %= 60; temp.h = h + t.h + temp.m / 60; temp.m %= 60; return temp; } void show() { cout << h << ":" << m << ":" << s << endl; } }; int main() { Time t1(23, 45, 50), t2(22, 34, 59), t3; t3 = t1 + t2; t3.show(); return 0; } </pre>	
---	--

Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings

CODE	TESTCASES
-------------	------------------

```
#include <iostream>
#include <string>
using namespace std;
class STRING {
char s[100];

public:
    void input() {
```

EQUAL STRINGS

```

        cin >> s; // reads till
space      }      bool
operator==(const STRING& t) const {
return strcmp(s, t.s) == 0;
    }

    STRING operator+(const STRING& t)
const {
    STRING temp;

    // Ensure no overflow
    if (strlen(s) + strlen(t.s)
    >= 100) {          cout
    << "String overflow!" <<
    endl;              temp.s[0]
    = '\0';            return
    temp;

    }

    strcpy(temp.s, s);
    strcat(temp.s, t.s);
    return temp;
    }      void display()
const {          cout << s
    << endl;
    }
};

int main() {
    STRING s1, s2, s3;
    cout << "Enter first string:
";    s1.input();
    cout << "Enter second string:
";    s2.input();
    if (s1 ==
s2)

```

```

Enter first string: oops
Enter second string: oops
Equal
Concatenated string: oopsoops

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

UNEQUAL STRINGS

```

Enter first string: rashu
Enter second string: costubh
Not Equal
Concatenated string: rashucostubh

```

```

...Program finished with exit code 0
Press ENTER to exit console.

```

<pre> cout << "Equal" << endl; else cout << "Not Equal" << endl; s3 = s1 + s2; cout << "Concatenated string: "; s3.display(); return 0; } </pre>	
---	--

Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

CODE

<pre> #include<iostream> using namespace std; class Matrix { int a[10][10], r, c; public: void input() { cout << "Enter rows and cols: "; cin >> r >> c; for(int i = 0; i < r; i++) for(int j = 0; j < c; j++) cin >> a[i][j]; } void display() { for(int i = 0; i < r; i++) { for(int j = 0; j < c; j++) cout << a[i][j] << " "; cout << endl; </pre>

```
}  
}
```

```
friend Matrix operator*(Matrix, Matrix);
```

```

};

Matrix operator*(Matrix m1, Matrix m2) {
Matrix m3;      m3.r = m1.r;      m3.c =
m2.c;
    for(int i = 0; i < m3.r; i++) {
for(int j = 0; j < m3.c; j++) {
m3.a[i][j] = 0;
        for(int k = 0; k < m1.c; k++)
            m3.a[i][j] += m1.a[i][k] * m2.a[k][j];
    }
}
    return m3;
}

int main() {
    Matrix m1, m2, m3;      cout
<< "Matrix 1" << endl;
m1.input();
    cout << "Matrix 2" << endl;
m2.input();      m3 = m1 * m2;
    cout << "Result" << endl;
m3.display();      return 0;
}

```

TESTCASE

```
Matrix 1:  
Enter rows and columns: 2  
2  
Enter elements:  
1  
2  
3  
4  
Matrix 2:  
Enter rows and columns: 2  
2  
Enter elements:  
4  
5  
6  
7  
Result:  
16 19  
36 43  
  
...Program finished with exit code 0  
Press ENTER to exit console.█
```

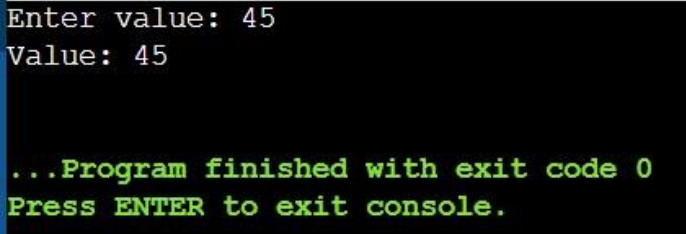
Overload '[]' to check array index out of bounds problem at run time.

CODE	TESTCASES
<pre>#include<iostream> using namespace std; class Array { int a[10]; public: void input() { for(int i = 0; i < 10; i++) cin >> a[i]; } int operator[](int index) { if(index < 0 index >= 10) { cout << "Index out of bounds" << endl; return -1; } return a[index]; } }; int main() { Array arr; arr.input(); int i; cout << "Enter index: "; cin >> i; cout << arr[i] << endl; return 0; }</pre>	VALID INDEX ELEMENT RETURNED <pre>Enter array elements (MAX 5)34 21 12 5 55 Enter index: 3 5 ...Program finished with exit code 0 Press ENTER to exit console.</pre> INVALID INDEX ERROR THROWN <pre>Enter array elements (MAX 5)4 21 33 2 1 Enter index: 6 Index out of bounds -1 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

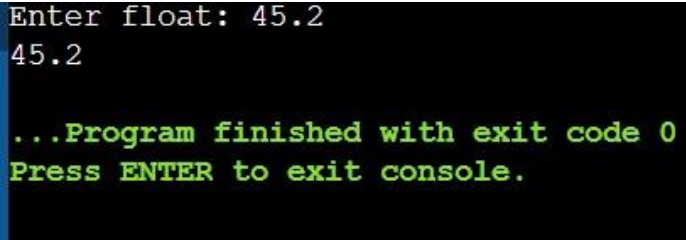
Overload '()' to input arbitrary number of input arguments for an object.

CODE	TESTCASES
<pre>#include<iostream> using namespace std; class Test { public: void operator()(int a, int b, int c) { cout << "Sum = " << a + b + c << endl; } }; int main() { Test t; t(10, 20, 30); return 0; }</pre>	<pre>Sum = 60 ...Program finished with exit code 0 Press ENTER to exit console.</pre>

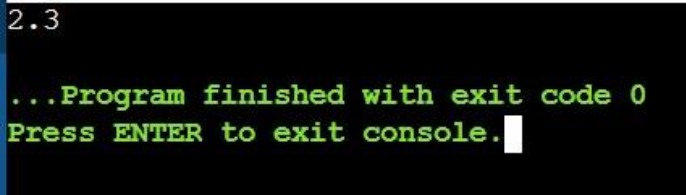
Write a program in C++ to overload input operator (>>) and output operator (<<).

CODE	TESTCASES
<pre>#include<iostream> using namespace std; class Test { int x; public: friend istream& operator>>(istream &in, Test &t); friend ostream& operator<<(ostream &out, Test &t); }; istream& operator>>(istream &in, Test &t) { in >> t.x; return in; }</pre>	 The screenshot shows a terminal window with a black background. The first line is 'Enter value: 45' in white text. The second line is 'Value: 45' in white text. The third line is '...Program finished with exit code 0' in green text. The fourth line is 'Press ENTER to exit console.' in green text.
<pre>ostream& operator<<(ostream &out, Test &t) { out << t.x; return out; } int main() { Test t; cout << "Enter value: "; cin >> t; cout << "Value: " << t << endl; return 0; }</pre>	

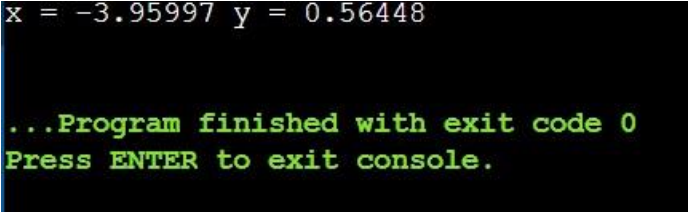
Write a program to convert basic data type (float) to user defined data type (object).

CODE	TESTCASES
<pre>#include<iostream> using namespace std; class Test { float x; public: Test(float f) { x = f; } void show() { cout << x; } }; int main() { float f; cout << "Enter float: "; cin >> f; Test t = f; t.show();</pre>	
<pre> return 0; }</pre>	

Write a program to convert UDT to basic data type (float)

CODE	TESTCASES
<pre>#include<iostream> using namespace std; class Test { float x; public: Test(float a) { x = a; } operator float() { return x; } }; int main() { Test t(5.5); float f = t; cout << f; return 0; }</pre>	 A screenshot of a terminal window with a black background and green text. The first line shows the output '2.3'. The second line shows the message '...Program finished with exit code 0' followed by 'Press ENTER to exit console.' and a white cursor icon at the end of the line.

How will you convert one UDT to another UDT. For example conversion of polar to cartesian system

CODE	TESTCASES
<pre>#include<iostream> #include<math.h> using namespace std; class Cartesian; class Polar { float r, theta; public: Polar(float a, float b) { r = a; theta = b; } operator Cartesian(); }; class Cartesian { float x, y; public: Cartesian() {} Cartesian(float a, float b) { x = a; y = b; } void show() { cout << "x = " << x << " y = " << y << endl; } }; Polar::operator Cartesian() { float x = r * cos(theta); float y = r * sin(theta); return Cartesian(x, y); } int main() { Polar p(2,3); Cartesian c = p; c.show(); return 0;</pre>	

}	